

001

# Multiprogramming

Operating Systems

# Multiprogramming

## Building a Bare-Metal ARM Operating System

### 1. General Description

This project implements a **minimal multiprogramming (multitasking) system** on the **BeagleBone Black** (AM335x, ARM Cortex-A8). The system runs **without an underlying OS** and manages **three distinct programs** resident in memory:

1. **OS block** — Responsible for hardware initialization, interrupt handling, process scheduling, and context switching.
2. **User Process 1 (P1)** — Prints digits **0** through **9** in a loop, then repeats.
3. **User Process 2 (P2)** — Prints lowercase letters **a** through **z** in a loop, then repeats.

Because there is **no MMU** (Memory Management Unit), all programs are loaded at **fixed memory addresses** defined at link time. The OS uses the AM335x **DMTimer2** to generate **periodic interrupts** (e.g., every second). On each timer interrupt, a **Round-Robin scheduler** performs a **context switch** between the two user processes. Each process runs for one time slice, then is preempted; the result is **interleaved output** (numbers and letters alternating over time), demonstrating that multiple processes are sharing the CPU.

The architecture follows a **layered approach**:

- **Low-level (OS):** `os.c` and `root.s` — hardware init (UART, timer, interrupt controller), vector table, interrupt handler, context switch, **PCB management**.
- **Library:** `stdio.c`, `string.c`, `os.c` (I/O) — provide PRINT (and optionally READ) and string utilities so user code does not touch hardware directly.
- **User programs:** Two processes (e.g., `P1/main.c`, `P2/main.c`) — implement only the letter/digit printing logic and use the library interface.

### 2. Project Objectives

By the end of this project you will have designed and implemented a bare-metal multiprogramming system that:

1. **Loads three programs** (OS and two user processes) into **fixed, non-overlapping memory regions**.
2. **Initializes the AM335x hardware:** UART for serial I/O, DMTimer2 for periodic interrupts, INTC (interrupt controller) to route timer interrupts, and (importantly) **disables the watchdog timer** to prevent automatic board reset.

3. **Implements a Round-Robin scheduler** driven by timer interrupts, switching between the two user processes every time slice.
4. **Performs correct context switching:** saves and restores all relevant process state (registers, stack pointer, program counter, link register, and status) so that each process resumes exactly where it was interrupted.
5. **Gives each process its own stack** so that their execution does not overlap or corrupt each other.
6. **Produces interleaved output** on the serial console (numbers from P1 and letters from P2), demonstrating successful multitasking.

The system should be **robust** (no crashes or hangs), with **clear separation** between OS, library, and user code, and **predictable timing** (e.g., one time slice per process per timer period).

### 3. Assignment Tasks

#### 3.1. Hardware Initialization (OS Block)

- **root.s (assembly)**
  - Set CPU to System mode (or appropriate privileged mode).
  - Set the **initial stack pointer** for the OS (e.g., to the top of the OS stack region).
  - **Clear the .bss section** (from `__bss_start__` to `__bss_end__`) to zero; this is required so that global variables (e.g., the PCB array) start in a known state.
  - Optionally place a **memory barrier** (e.g., `dsb`; `isb`) after clearing `.bss` so that all writes are visible before entering C.
  - Set up the **ARM vector table** (e.g., via `VBAR`, `CP15 c12`) so that the IRQ vector points to your IRQ handler.
  - Jump to the C main function (e.g., `bl main`).
- **os.c (or equivalent C init)**
  - **Disable the watchdog timer** (WDT1 on AM335x) as early as possible (e.g., first thing in main), using the required write sequence to the WDT registers; otherwise the board will reset after about 60 seconds.
  - **UART0** (base `0x44E09000`): ensure it is usable for serial output (transmit holding register, line status register). The bootloader may have already initialized it; your code must at least use the correct base address and poll `THRE` before sending.
  - **DMTimer2** (base `0x48040000`): configure for periodic interrupts (e.g., 1 s). Set load value, enable overflow interrupt (TIER), clear status (TISR), and start the timer with auto-reload (TCLR).

- **INTC** (base 0x48200000): reset if needed, then **unmask IRQ 68** (DMTimer2 overflow) so that the CPU receives the timer interrupt.
- **Enable IRQ** in the CPU (clear the I-bit in CPSR) so that timer interrupts are taken.
- **Vector table and IRQ handler**
  - Implement an **IRQ handler** in assembly that: saves R0–R12 and LR\_irq (the return address) to a known location (e.g., global arrays), calls a C function `timer_irq_handler()`, then restores R0–R12 and LR\_irq and returns from interrupt (e.g., `subs pc, lr, #4`).
  - The C function will use the saved values to update the current process's PCB and restore the next process's context.

### 3.2. Memory Layout and Linking

- **Design a fixed memory map** so that the OS and the two user processes do not overlap. Example layout (addresses can vary; the following is consistent with a typical solution):

| Region       | Start      | Size  | Description                  |
|--------------|------------|-------|------------------------------|
| OS code/data | 0x82000000 | 64 KB | OS .text, .data, .bss        |
| OS stack     | 0x82010000 | 8 KB  | OS stack (grows down)        |
| P1 code/data | 0x82100000 | 64 KB | Process 1 .text, .data, .bss |
| P1 stack     | 0x82110000 | 8 KB  | Process 1 stack              |
| P2 code/data | 0x82200000 | 64 KB | Process 2 .text, .data, .bss |
| P2 stack     | 0x82210000 | 8 KB  | Process 2 stack              |

You may use different base addresses (e.g., 0x80000000) as long as they respect the AM335x memory map (e.g., avoid peripheral regions) and do not overlap.

- **Linker scripts**
  - Provide a **linker script** for the OS that places code and data at the OS base and reserves the OS stack region.
  - Provide **linker scripts** for P1 and P2 so that each is linked to run at its own base address and has its own stack region.
- **Build**
  - The OS is built as one image (e.g., `root.s + os.c + ...`) linked for the OS base.
  - P1 and P2 are built **separately** as standalone images linked at their respective bases.

- **No dynamic loading** is required: P1 and P2 are loaded into memory by the bootloader (or by you) at their fixed addresses before starting the OS; the OS then schedules their entry points (see below).

### 3.3. Process Control Blocks (PCB)

- Define a **PCB structure** that holds everything needed to save and restore a process. At minimum:
  - **Process ID** (e.g., 0 = OS, 1 = P1, 2 = P2).
  - **Stack pointer (SP)** — value of the process's stack pointer (e.g., R13 in SVC mode) when it was interrupted.
  - **Program counter (PC)** — address of the instruction to resume.
  - **Link register (LR)** — R14 in SVC mode (return addresses for the process).
  - **Saved Program Status Register (SPSR)** — copy of CPSR when the process was interrupted (optional but useful for correct mode/state restore).
  - **General-purpose registers** — R0–R12.
  - **State** (optional but recommended) — e.g., RUNNING, READY — for clarity and future extension.
- Maintain an **array of PCBs** (e.g., one for OS, one for P1, one for P2). The **OS** initializes each PCB with the process's **entry point (PC)**, **initial stack pointer** (top of that process's stack), and initial register values (e.g., 0).

### 3.4. Stack Setup for Each Process

- Each process must have its **own stack** (separate memory region). Before the first time a process runs, the OS must set up an **initial stack frame** so that when the process is “restored” from its PCB, it has a valid SP and the correct PC (and optionally LR) to start execution.
- Typical approach:
  - Reserve a contiguous region for the process stack (e.g., 8 KB).
  - Build a **saved context** at the top of that region: e.g., 14 words (R0–R12 and LR), with **LR set to the process entry point** (so that when the “return from interrupt” logic restores LR and then jumps, the process starts at its main).
  - Set the process's **SP** in its PCB to the address of the lowest word of this frame (e.g., where R0 would be stored).
  - Set the process's **PC** in the PCB to the entry point (e.g., 0x82100000 for P1, 0x82200000 for P2).
- Ensure stacks **do not overlap** with each other or with code/data.

### 3.5. Context Switching

- **On each timer interrupt:**
  - a. **Acknowledge** the timer interrupt (clear the timer status bit) and send **End of Interrupt** to the INTC so that further IRQs can be taken.
  - b. **Save the current process context:**
  - c. **Choose the next process** in Round-Robin order (e.g., if current is P1, next is P2; if current is P2, next is P1).
  - d. **Restore the next process's context:**
  - e. Return from the assembly IRQ handler (restore R0–R12 and LR, then subs pc, lr, #4). The CPU will then resume the **next** process at its saved PC.
- **First run of a user process:**
  - The OS runs first. After initializing hardware, PCBs, and stacks, it performs an **initial context switch** to P1 (or P2).
  - For this first switch, there is no “previous” saved state; the PCB already holds the entry point and the initial stack pointer. The OS can either use the same restore path (with PCB values set as above) or use a small code path that sets SVC SP and LR (or PC) and jumps to the process entry point (e.g., movs pc, lr with LR = entry point).

### 3.6. User Process Implementation

- **Process 1 (P1)**
  - Implement in a file such as P1/main.c.
  - Use the library PRINT (e.g., PRINT("----From P1: %d\n", n);) to avoid direct UART access.
  - In an infinite loop: print digits **0** to **9** (then wrap to 0), and optionally add a short delay loop so that output is readable and the process does not dominate the CPU between timer ticks.
- **Process 2 (P2)**
  - Implement in a file such as P2/main.c.
  - Use PRINT (e.g., PRINT("----From P2: %c\n", c);).
  - In an infinite loop: print letters **a** to **z** (then wrap to 'a'), and optionally add a short delay.
- Both processes run **indefinitely**; they do not exit. Control is transferred between them **only** by the timer-driven context switch.

### 3.7. Build and Load

- **Build:**
  - Compile and link the **OS** into one binary (e.g., os.bin or os.elf) linked at the OS base address.
  - Compile and link **P1** and **P2** into two separate binaries, each linked at its own base address.
- **Load (e.g., via U-Boot)**
  - Load the OS image to the OS base (e.g., loady 0x82000000 then send the file).
  - Load the P1 image to the P1 base (e.g., loady 0x82100000).
  - Load the P2 image to the P2 base (e.g., loady 0x82200000).
  - Start execution at the **OS** entry point (e.g., go 0x82000000). The OS will initialize and then switch to P1 (or P2); from then on, the timer will alternate between P1 and P2.
- Document the exact **load addresses** and **go** command (e.g., in a Loads.txt or README) so that the system can be reproduced.

## 4. Expected Output and Verification

- **Serial console** should show **interleaved** lines from P1 and P2, for example:

```

----From P1: 0
----From P1: 1
...
----From P2: a
----From P2: b
...
----From P1: 3
----From P2: d
...

```

The exact interleaving depends on timer period and process run time; the important point is that **both** processes produce output over time, demonstrating that the scheduler is switching between them.

- **Verification:**
  - Confirm that **both** P1 (digits) and P2 (letters) appear.
  - Confirm that the board **does not reset** after ~60 s (watchdog disabled).
  - Confirm that execution continues **without hang** (correct context save/restore and return address handling).

## 5. Implementation Hints

- **Watchdog:** Disable WDT1 very early in main using the AM335x sequence (e.g., write 0xAAAA then 0x5555 to WDT\_WSPR, waiting for WDT\_WWPS between writes). Otherwise the board will reset periodically.
- **.bss:** Clear .bss in root.s before calling main. Follow with a **memory barrier** (dsb; isb) so that all cleared memory is visible before C code runs.
- **Memory barriers:** After initializing the PCB array (and other critical shared state) in C, use a **DSB** (e.g., `asm volatile ("dsb" ::: "memory");`) so that all PCB writes are visible before the first context switch or interrupt. This avoids subtle bugs on ARM where writes can be reordered or delayed.
- **Return address (PC):** When an IRQ occurs, the CPU saves a return address in LR\_irq (often PC+8 or similar). When saving the current process, compute **PC = LR\_irq - 4** (or the correct offset for your core) so that you store the **instruction that was interrupted**. When restoring, set **LR\_irq = saved\_PC + 4** so that `subs pc, lr, #4` returns to that instruction.
- **SVC mode SP/LR:** The interrupt runs in IRQ mode; the process was in SVC mode. To save/restore the process's SP and LR, temporarily switch to SVC mode (with IRQs disabled), read or write SP and LR, then switch back to IRQ mode and re-enable IRQs.
- **Stack frame layout:** The initial stack frame for a process (for the first run) should match what your restore path expects. A common layout is 14 words: R0–R12 and LR, with SP pointing to the R0 slot and LR containing the **entry point** so that when you “return” to the process, it starts at its main.
- **Timer:** Use a known-good DMTimer2 configuration: TCLR with auto-reload (e.g., 0x3), TIER overflow enable (e.g., 0x2), and correct TLDR for the desired period (e.g., ~1 s at 24 MHz). Unmask IRQ 68 in the INTC (e.g., `INTC_MIR_CLEAR2`, bit 4).
- **Debugging:** Add minimal UART output in the timer handler (e.g., “Tick” or current PID) only if needed; avoid heavy printing inside the handler to keep timing predictable. Use a serial console (e.g., screen or minicom) to capture output.
- **Safety:** Disable interrupts (set I-bit) during the critical section of the context switch (when updating SP/LR in SVC mode) to avoid reentrancy.



## 6. Optional / Extra Points

- **Process state:** Add a **state** field (e.g., RUNNING, READY) in the PCB and update it when saving/restoring context; this prepares for future blocking or multiple ready processes.
- **Voluntary yield:** Implement a **yield** function (e.g., a software trap or SVC) that a process can call to voluntarily give up the CPU; the handler would perform a context switch to the other process. This demonstrates cooperative switching in addition to timer-driven preemption.
- **Three contexts:** Some solutions also keep an “OS” context (e.g., a simple loop that prints something) and rotate among OS, P1, and P2; the core requirements are still two user processes (P1 and P2) with Round-Robin and correct context switch.